

Week 7 Lab: SQL Injection Attacks

SQL injection remains one of the most reliably exploited vulnerabilities in database-driven applications, consistently ranked in the OWASP Top 10 because it requires no specialized tooling and produces immediate, high-value results for an attacker (OWASP 2021). The core technique involves inserting SQL syntax into a user-controlled input field so that the input is interpreted as query logic rather than a data value. In this lab, the vulnerable query concatenated the @input variable directly into the WHERE clause, which meant the attacker-controlled string `OR 1=1` rewrote the condition to be unconditionally true. As shown in Figure 2, the injected query returned 19,972 rows from `Person.Person`, the entire table, rather than the 103 rows a legitimate Smith search produced in Figure 1. The Messages tab confirmed the exact SQL that executed, making the mechanism visible: `WHERE LastName = '' OR '1' = '1'`.

The primary defence demonstrated in this lab is parameterized query execution via `sp_executesql`. Rather than concatenating input into the SQL string, the query template is fixed and the input value is passed as a typed parameter bound at execution time (Microsoft 2024). SQL Server treats the parameter value as a data literal regardless of its content, so the injection string `OR 1=1` is searched as a last name rather than parsed as SQL logic. Figure 6 shows zero rows returned when the injection input was run against the parameterized version, confirming the defence held. Additional techniques that should accompany parameterized queries in production include input validation and allowlisting to reject input that does not match expected patterns, stored procedures that accept typed parameters and expose no raw SQL construction to the application layer, least-privilege database accounts that limit what any injected query can access even if injection occurs, and Web

Application Firewall rules that detect and block common injection patterns at the network perimeter (OWASP 2021).

The most important thing I wish I had known before starting this lab is that the Messages tab in SSMS is as important as the Results tab. The Results tab shows what data was returned, but the Messages tab shows the SQL that actually executed, which is where the injection is visible. Without checking Messages, the attack in Step 2 looks like a successful query rather than a security failure. I also wish I had understood upfront why the T-SQL syntax for the injection string uses doubled single quotes: in T-SQL, a single quote inside a string literal is escaped by doubling it, so the code writes `OR '1' = '1` to produce the runtime string `OR '1' = '1'`. That distinction is not explained in the lab instructions and caused confusion before the query ran successfully.

References

Microsoft. (2024). *sp_executesql* (Transact-SQL).

<https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-executesql-transact-sql>

OWASP. (2021). *A03:2021 Injection*. https://owasp.org/Top10/A03_2021-Injection/

Figure 1: Vulnerable query with legitimate input – Results tab showing 103 Smith rows returned as expected

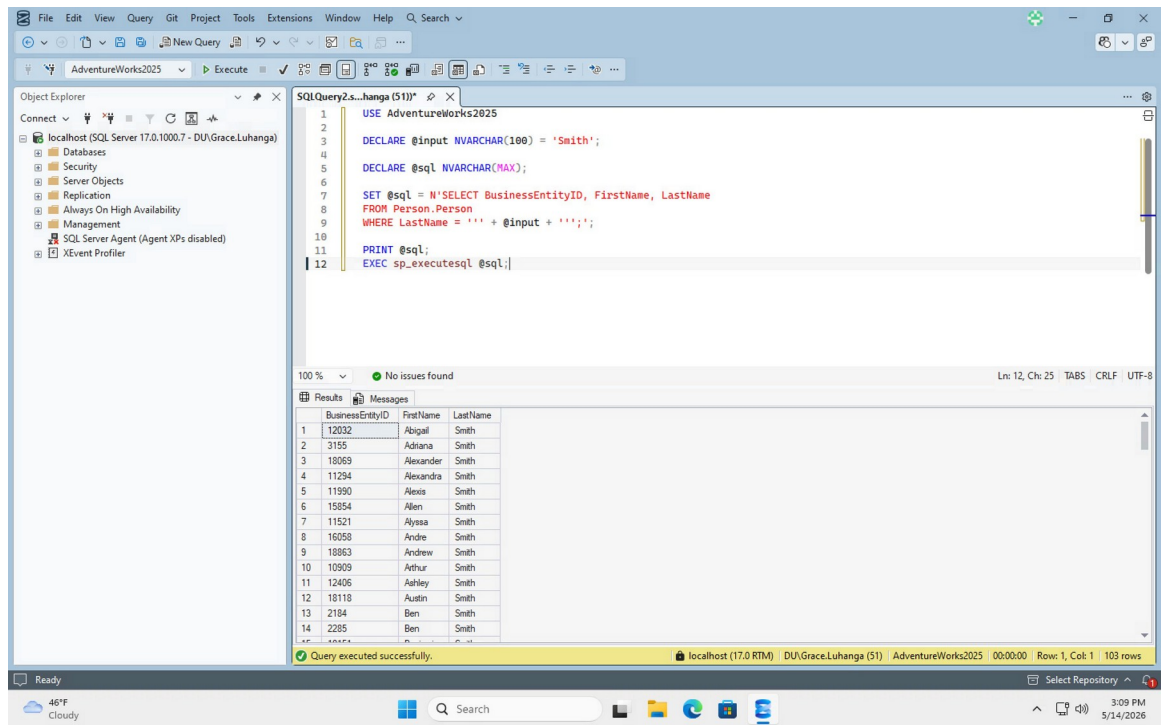


Figure 2: Vulnerable query with legitimate input – Messages tab confirming WHERE LastName = 'Smith', 103 rows affected



Figure 3: SQL injection attack – Results tab showing 19,972 rows returned after OR 1=1 injection against vulnerable query

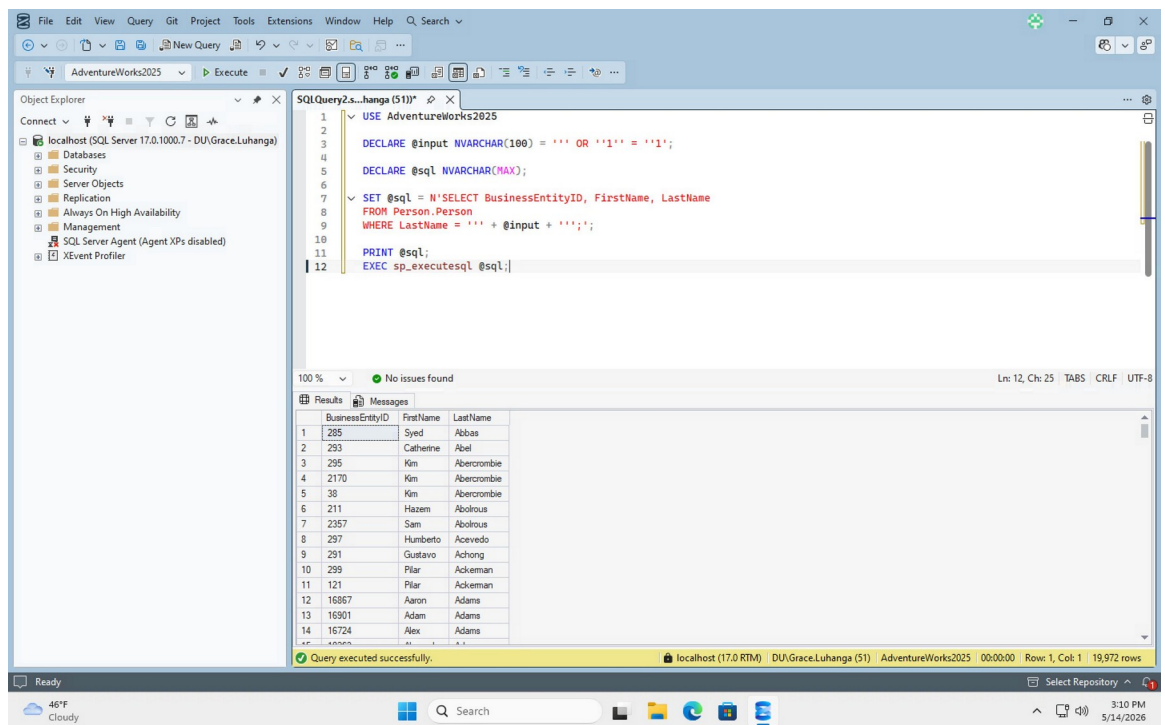


Figure 4: SQL injection attack – Messages tab confirming injected WHERE LastName = '' OR '1' = '1', 19,972 rows affected

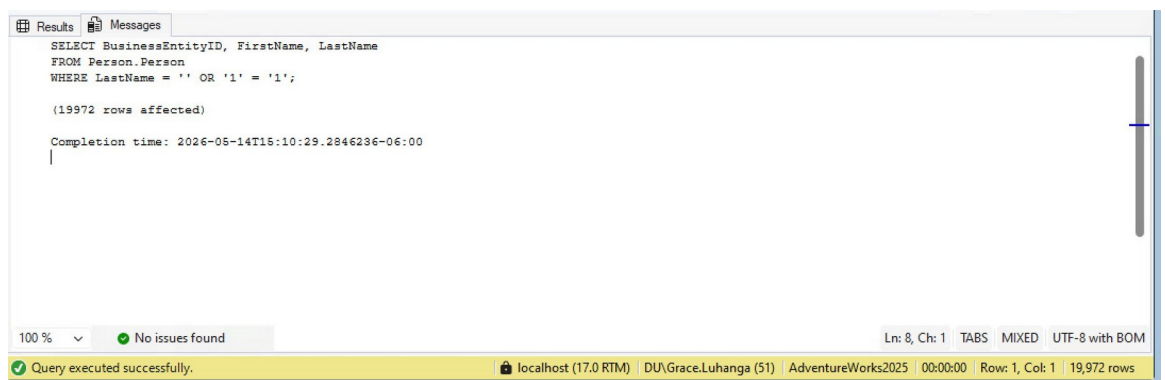


Figure 5: Parameterized query with legitimate input – Results tab showing top 10 Smith rows, injection surface eliminated

The screenshot shows the SQL Server Enterprise Manager interface. The query editor contains the following T-SQL code:

```
1 USE AdventureWorks2025
2
3 DECLARE @input NVARCHAR(100) = 'Smith';
4
5 DECLARE @safe_sql NVARCHAR(MAX) =
6 N'SELECT TOP (10) BusinessEntityID, FirstName, LastName
7 FROM Person.Person
8 WHERE LastName = @lname;';
9
10 PRINT @safe_sql;
11
12 EXEC sp_executesql @safe_sql, N'@lname NVARCHAR(100)', @lname = @input;
```

The Results tab displays the following data:

BusinessEntityID	FirstName	LastName
12032	Abigail	Smith
3155	Adriana	Smith
18069	Alexander	Smith
11294	Alexandra	Smith
11990	Alexis	Smith
15854	Allen	Smith
11521	Alyssa	Smith
16058	Andre	Smith
18863	Andrew	Smith
10909	Arthur	Smith

The status bar indicates "Query executed successfully." and "No issues found".

Figure 6: Injection attack against parameterized query – Results tab showing 0 rows returned, attack neutralized

The screenshot shows the SQL Server Enterprise Manager interface. The query editor contains the following T-SQL code:

```
1 USE AdventureWorks2025
2
3 DECLARE @input NVARCHAR(100) = '' OR '1' = '1';
4
5 DECLARE @safe_sql NVARCHAR(MAX) =
6 N'SELECT TOP (10) BusinessEntityID, FirstName, LastName
7 FROM Person.Person
8 WHERE LastName = @lname;';
9
10 PRINT @safe_sql;
11
12 EXEC sp_executesql @safe_sql, N'@lname NVARCHAR(100)', @lname = @input;
```

The Results tab is empty, showing 0 rows returned. The status bar indicates "Query executed successfully." and "No issues found".

Figure 7: Injection attack against parameterized query – Messages tab confirming WHERE LastName = @lname, 0 rows affected

